Threat Model Review for Financial System on GCP

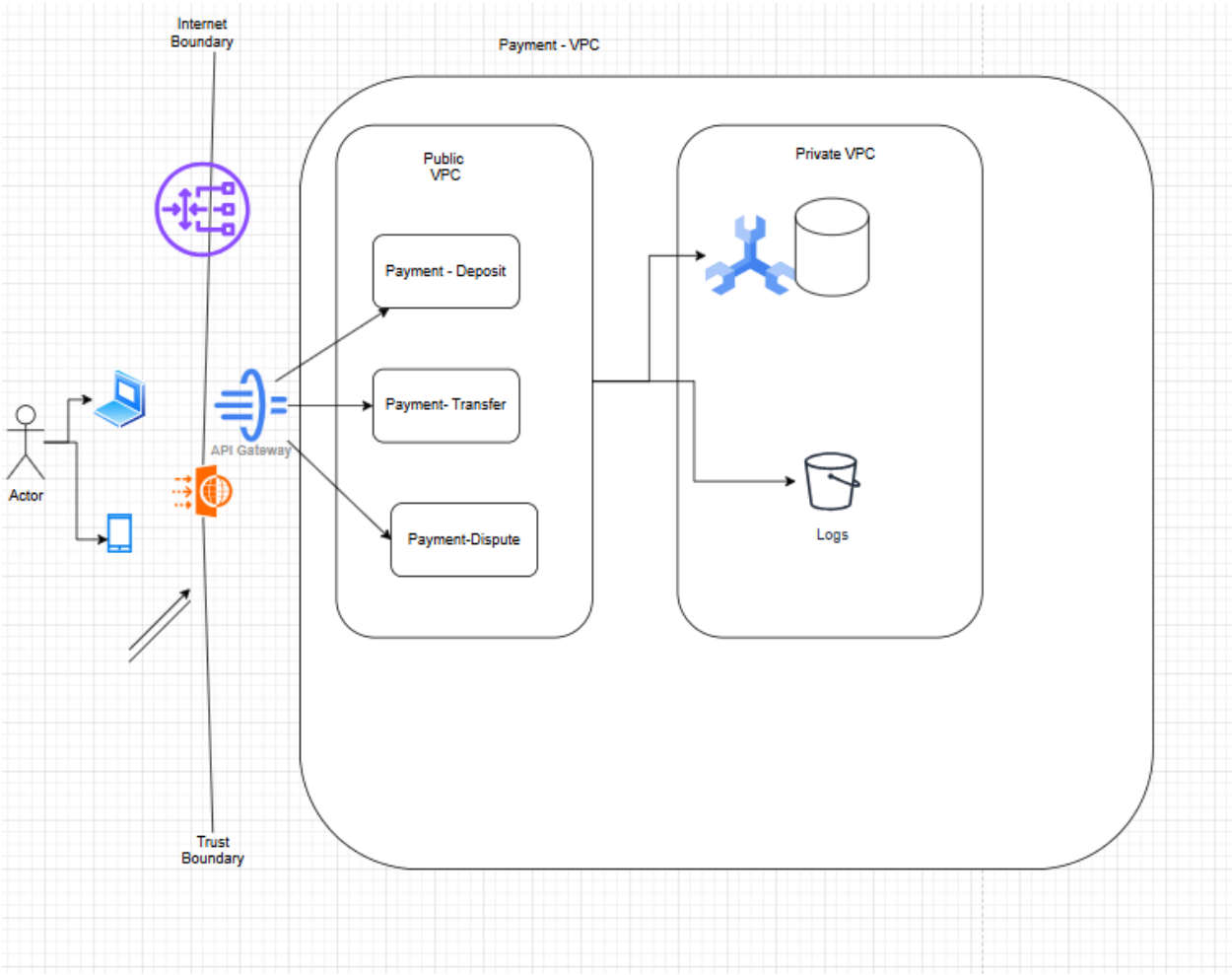
Problem Statement:

You are designing a new microservices platform on GCP that processes financial transactions. Services communicate over internal APIs, data is stored in Cloud Spanner, and the platform is accessed by mobile clients over the public internet. Walk us through how you would threat model this system.

Introduction:

This threat model analyzes a GCP-based microservices platform processing financial transactions, with three core services planned to go live: Payment Deposit, Payment Transfer, and Payment Dispute. It identifies potential security threats across the system, including client interactions, internal service communication, and data storage in Cloud Spanner, and outlines mitigation strategies to ensure confidentiality, integrity, and availability of financial data.

High level Architecture diagram shared by development team



Asset	Descriptions	Impact	Notes
Customer PII data	Personally identifiable information such as name, email, phone, address	Critical	Critical for privacy and regulatory compliance (e.g., GDPR, PCI DSS); must be encrypted at rest and in transit
API gateway	Cloud Endpoints / Apigee managing all client requests	High	Entry point for all traffic; enforces authentication, rate limiting, and access control; compromise can lead to unauthorized access
Customer's Financial Data Database	Cloud Spanner storing account balances, transaction history	Crical	Core financial data; integrity, confidentiality, and availability are critical; access must be strictly controlled and audited

DataFlow Diagram

Threat Model

Threat Vector	Description	Impact & Asset	Security Controls Service team need to implement	Stat tus
Authentication (S - Spoofing)	Threat: A malicious actor spoofs a legitimate user identity or hijacks an active session to bypass perimeter defenses. By presenting forged or stolen credentials to the public-facing API gateway, the attacker gains unauthorized access to perform state-changing operations (GET/PUT). Impact: This lead to the compromise of financial transaction integrity, unauthorized payment initiation, and the exfiltration of sensitive customer financial data, resulting in significant regulatory and financial liability.	High	*Governing Principles: Secure by Default; Principle of Least Privilege* 1. Implement robust Federated Identity using OIDC, OAuth 2.0, or SAML 2.0 to ensure cryptographically verifiable identities. 2. Adopt a "Fail-Secure" architecture where all requests are denied by default unless explicitly authenticated and authorized. 3. Enforce Multi-Factor Authentication (MFA) for	Reve iw

			all administrative, high-privilege, and sensitive transaction-based accounts. 4. Eliminate "open" endpoints by requiring valid credentials for all microservices and API routes. 5. Issue ephemeral, short-lived access tokens (e.g., JWTs with limited TTL) to minimize the blast radius of token theft. 6. Digitally sign tokens using strong asymmetric algorithms (e.g., RS256 or ES256) and perform mandatory signature validation on every request.	
Authorization (Elevation of Privileges)	Attack Vector (Authorization): An attacker exploits weak or missing authorization checks (e.g., missing object-level checks) in the API to perform actions or access data they should not be permitted to, potentially leading to privilege escalation. Impact: Unauthorized access to customer accounts or financial transactions, unauthorized state changes (e.g., initiating fraudulent transfers), and data exfiltration.	High	7. Governing Principle: Principle of Least Privilege (PoLP) 8. 1. Policy Enforcement: Implement robust authorization (AuthZ) checks at the API gateway and within microservice business logic, ensuring fine-grained, object-level authorization (e.g., users can only operate on their own financial records). 9. 2. IAM Roles: Grant users, microservices, and service accounts only the absolute minimal, fine-grained IAM roles required. Strictly avoid broad roles like Owner or Editor. 10. 3. Identity Security: Limit service account permissions and enforce key rotation. Utilize Workload Identity Federation to manage	Reve iw

			service identity without relying on static keys. 11. 4. Continuous Verification: Create automated Canary tests or integration tests to continuously verify that authorization logic (AuthZ) remains effective across all deployment stages. 12.	
DOS - Denial of Service	Threat: A malicious external actor attempts to overwhelm the public-facing API gateway and microservices by flooding them with high-volume or application-layer HTTP requests. Impact: The primary impact is the loss of Availability (unresponsive services), preventing legitimate customers from initiating transactions. For a financial system, prolonged unavailability can lead to severe reputational d amage and financial losses.	High	13. Governing Principle: Availability and Resilience 14. 15. 1. Network-level DDoS Protection: Utilize Google Cloud's built-in network DDoS mitigation to protect against volumetric Layer 3 and Layer 4 attacks. 16. 2. WAF and Rate Limiting: Implement a Web Application Firewall (WAF), such as Cloud Armor, at the API Gateway to enforce Layer 7 (application) rate limiting, throttle suspicious traffic, and implement Geo-blocking to restrict access from unwanted regions. 17. 3. Auto-Scaling and Load Balancing: Configure Cloud Load Balancer and auto-scaling for microservices to dynamically absorb traffic spikes and distribute load effectively, ensuring the system remains responsive under load. 18. 4. Bot Mitigation: Employ advanced bot defense mechanisms (e.g., reCAPTCHA Enterprise) on	Reve iw

			sensitive or high-velocity endpoints to differentiate between legitimate user interaction and automated scripts.	
Data In Transit (TLS/mTLS) - Data Protection Tampering, Information Disclosure Tampering, Information Disclosure	Threat: External or internal actors intercept data in transit between mobile clients and the API Gateway, or between internal microservices, to eavesdrop on PII and financial transaction details. Impact: Loss of confidentiality and integrity of financial data, leading to fraudulent transactions and regulatory non-compliance.	High	19. Enforce TLS 1.2/1.3 for all public endpoints using Google Cloud Armor and Cloud Load Balancing. 20. Implement mutual TLS (mTLS) for secure service-to-service communication within the VPC using Anthos Service Mesh or Istio. 21. Standardize on strong ciphers (e.g., AES-256) and deprecate weak or legacy suites. 22. Utilize VPC Service Controls to create a service perimeter, mitigating risks of data exfiltration to unauthorized GCP projects.	Reve iw
Data at Rest - Data Protection Information Disclosure, Tampering Tampering	Threat: An adversary gains unauthorized access to the underlying storage of Cloud Spanner or backups to exfiltrate customer account balances and PII. Impact: Massive data breach, severe reputational damage, and significant financial penalties under GDPR/PCI DSS.	High	23. Implement Customer-Managed Encryption Keys (CMEK) via Cloud KMS for granular control over data encryption in Cloud Spanner. 24. Apply the Principle of Least Privilege (PoLP) using IAM conditions to restrict access to sensitive datasets based on resource tags or request attributes. 25. Store and rotate application secrets, such as database credentials, using Secret Manager with automatic rotation enabled. 26. Enable Cloud Data Loss Prevention (DLP) to	Reve iw

			automatically scan, discover, and redact sensitive PII before storage.	
Input Validation Tampering API Abuse, SSRF, Micro services communication	● An attacker sends unexpected, malformed, or malicious input (e.g., SQL, NoSQL, JSON, XML, scripts) to APIs or microservices. ● This can bypass business rules, manipulate transactions, inject code, or crash services. Impact: ● Unauthorized modification of financial transactions or data. ● Data corruption or loss. ● Potential execution of malicious scripts or injection attacks.	High	27. Review API designs and specifications to ensure proper authentication, authorization, data exposure limits, and error handling before deployment. Prevents misuse and reduces attack surface. 28. Only accept expected characters, formats, and types (e.g., numeric amounts, specific date formats). 29. Block known harmful characters or patterns (e.g., <script>, SQL meta-characters).	Reve iw
Infrastructure as Cloud Network Access control (VPC, NACLs, Security Groups) Spoofing, Tampering	Misconfigurations or vulnerabilities in Infrastructure as Code templates (e.g., Terraform, Cloud Deployment Manager) can unintentionally expose cloud resources, grant excessive permissions, or create insecure networking and storage configurations. An attacker who exploits these misconfigurations could gain unauthorized access to microservices, databases, or sensitive customer data, disrupt services, or exfiltrate financial information.	High	30. 1.Use private VPCs to isolate internal services from the public internet. 31. 2.Create subnets per environment or microservice type (e.g., frontend, backend, database) to limit lateral movement. 32. 3. Enable VPC Flow Logs for all subnets to monitor traffic and detect anomalies. 33. 4.Restrict access to sensitive services (Cloud Spanner, Cloud Storage) to only trusted networks. 34. 5. Implement strict allow-list rules, only opening required ports and protocols.	Reve iw

			35. 6. Use Cloud NAT for outbound internet access from private subnets without exposing instances publicly. 36. 7. Use VPC peering or Private Service Connect for secure service-to-service or cross-project communication.	
Logging, Monitoring & Auditing Repudiation	Threat: An adversary (external or malicious insider) gains unauthorized access to logging and monitoring systems (like Cloud Logging) to delete, modify, or disable critical audit trails and transaction logs. Impact: Concealing these activities results in "Repudiation," making it impossible to reliably prove or attribute fraudulent transactions or unauthorized access. This severely compromises system accountability and regulatory compliance.	High	37. Configure Cloud Storage with bucket lock and retention policies for critical logs to ensure immutability and prevent modification or deletion. 38. Implement strict IAM least privilege for log access (e.g., read-only roles) and log sink modifications. 39. Export critical Admin Activity Audit Logs to a secured, separate project sink for long-term, tamper-proof storage. 40. Configure alerts on any attempts to disable logging, modify log sinks, or delete audit logs	Reveiw
Business Logic Fraud / Sensitive Business Logic Tampering (Tampering/Availability):	Threat: An attacker exploits logic flaws, race conditions, or sequencing errors in financial transactions (e.g., Payment Deposit/Transfer) to manipulate account balances, bypass limits, or execute fraudulent refunds. Impact: This results in direct financial loss, corruption of ledger integrity, and potential regulatory fines due to unauthorized funds movement.	Critical (Financial Data Integrity)	41. 1. Server-Side State Validation: Implement strong server-side validation and state checks for all financial workflows; never trust client-side data or sequencing. 42. 2. Transactional Integrity: Utilize Cloud Spanner's strong consistency guarantees to prevent race conditions and ensure transactions are atomic.	Reveiw

			43. Workflow Enforcement: Implement a centralized workflow engine or state machine to enforce the correct sequence of operations and reject out-of-order API calls. 44. Limits & Thresholds: Implement automated checks for transaction limits, velocity, and abnormal transfer patterns.	
Supply Chain & Deployment Vulnerabilities (Spoofing/Tampering/Elevation):	Threat: Malicious or vulnerable components (e.g., open-source libraries, base container images, third-party code) are introduced during development or the CI/CD pipeline (e.g., Cloud Build) is compromised to inject backdoors. Impact: Unauthorized code execution, persistent access for attackers, and widespread security compromise across all deployed microservices.	High (System Integrity/ Availability)	45. 1. Vulnerability Scanning: Use tools like Container Analysis to scan all container images and dependencies for known vulnerabilities before deployment. 46. 2. Binary Authorization: Enforce policies to ensure only signed, verified container images from approved CI/CD pipelines can be deployed to GKE or Cloud Run. 47. 3. Dependency Management: Maintain an AIBOM/SBOM (Bill of Materials) for all components and pin dependencies to specific, verified versions. 48. 4. CI/CD Pipeline Hardening: Apply Least Privilege to CI/CD service accounts and use VPC Service Controls to restrict pipeline access to external resources.	Reveiw
Mobile Client Security (Spoofing/Information Disclosure/Tampering)	Threat: Attackers reverse-engineer the mobile application to uncover API endpoints, hardcoded keys, or sensitive business logic, or exploit weak local storage to steal authentication tokens. Impact: Account takeover (Spoofing) due to token theft and	High (Authentication/PII)	49. 1. Code Hardening: Implement code obfuscation, anti-debugging, and runtime integrity checks in the mobile application.	Reveiw

	unauthorized access to API endpoints (Tampering/Disclosure).		50. 2. Secure Storage: Use OS-native secure storage (e.g., iOS Keychain, Android Keystore) for storing credentials, tokens, and encryption keys. 51. 3. Certificate Pinning: Implement certificate pinning on the mobile client to prevent MITM attacks on the public API gateway. 52. 4. Server-Side Logic Shift: Ensure all critical authorization and business logic checks reside exclusively on the server (API Gateway/Microservices).	
Secrets Management & Key Compromise	Threat: An attacker, external or internal, compromises the Cloud KMS or Secret Manager services (or the high-privilege service accounts accessing them) to exfiltrate or tamper with database credentials, API keys, or Customer-Managed Encryption Keys (CMEK). Impact: Total loss of data confidentiality and integrity, system-wide service disruption, and immediate regulatory breach.	Critical (Confidentiality/Integrity)	53. Perimeter Control: Use VPC Service Controls to create a secure boundary around Cloud KMS and Secret Manager APIs, restricting access to trusted networks. 54. Strict IAM on Secrets: Apply the Principle of Least Privilege (PoLP) and IAM Conditions to restrict which microservices can access which specific secrets (secret-level granularity). 55. Workload Identity Federation: Use Workload Identity for all service-to-service authentication to eliminate the need for static service account keys. 56. Key Rotation & Auditing: Enforce automatic key rotation for all secrets and KMS keys, and configure alerts on any modification or failed access attempts.	Reveiw

Mitigation Controls Quick Reference Table

Security Domain	Technical Controls to Implement	Evidence with CRs for Controls
I. Identity & Access Management	• Implement robust Federated Identity using OIDC, OAuth, or SAML. • Enforce Multi-Factor Authentication (MFA) for high-privilege accounts. • Utilize Workload Identity Federation for service-to-service authentication. • Issue ephemeral, short-lived JWT access tokens. • Digitally sign and validate tokens using strong asymmetric algorithms (disregard, MD4, Sha128, RC4, 3DES..etc). • Grant minimal, fine-grained IAM roles to services and users. • Apply IAM Conditions for granular access control on data and secrets.	
II. Network & Perimeter Security	• Utilize VPC Service Controls to create a secure service perimeter around critical APIs (KMS, Secret Manager). • Use private VPCs and segment the network with subnets per microservice type (frontend, backend). • Implement Cloud Armor WAF for Layer 7 rate limiting, bot mitigation (reCAPTCHA Enterprise), and Geo-blocking. • Utilize Google Cloud's built-in network DDoS mitigation (Layer 3/4). • Implement strict network allow-list rules and use Cloud NAT for secure outbound internet access. • Use VPC peering or Private Service Connect for secure cross-project communication.	
III. Data Protection (Transit & Rest)	• Enforce TLS 1.2/1.3 for all public endpoints. • Implement mutual TLS (mTLS) for internal service-to-service communication. • Standardize on strong ciphers like AES-256. • Implement Customer-Managed Encryption Keys (CMEK) via Cloud KMS for Cloud Spanner. • Use Secret Manager for credential storage with automatic key rotation enabled. • Enforce automatic key rotation for all Secrets and KMS keys.	

Security Domain	Technical Controls to Implement	Evidence with CRs for Controls
	• Enable Cloud Data Loss Prevention (DLP) for PII scanning and redaction.	
IV. Application & Business Logic Security	• Implement strong server-side state validation and transactional integrity checks (leverage Cloud Spanner's strong consistency). • Implement a centralized workflow engine or state machine to enforce the correct sequence of API calls. • Implement automated checks for transaction limits, velocity, and abnormal patterns. • Strictly validate all input format, type, and character sets, and block known harmful characters/patterns.	
V. Deployment & Supply Chain	• Enforce Binary Authorization to ensure only signed, verified container images are deployed. • Use Container Analysis for scanning all images and dependencies for known vulnerabilities. • Maintain an AIBOM/SBOM (Bill of Materials) and pin dependencies to specific versions.	
VI. Logging & Observability	• Configure Cloud Storage with bucket lock and retention policies for log immutability. • Export critical Admin Activity Audit Logs to a secured, separate project sink. • Enable VPC Flow Logs for network traffic monitoring. • Configure alerts on attempts to disable logging, modify log sinks, or delete audit logs.	
VII. Mobile Client Security	• Implement certificate pinning on the mobile client to prevent MITM attacks. • Use OS-native secure storage (iOS Keychain, Android Keystore) for credentials. • Implement mobile code obfuscation, anti-debugging, and runtime integrity checks. • Shift all critical authorization and business logic checks exclusively to the server side.	